

CHAPTER TWO

THEORETICAL BACKGROUND

2.1 Introduction

The development of modern web applications has shifted significantly from static HTML pages to dynamic, interactive, and component-based single-page applications (SPAs). Building a complex platform like a Template-Based Website Builder requires a robust architecture capable of handling intensive operations such as real-time Document Object Model (DOM) manipulation, state management, and seamless client-server communication. This chapter provides a general overview of the foundational concepts, programming languages, tools, and database systems utilized to construct this project. It highlights the rationale behind selecting specific technologies over others to achieve high performance and full creative freedom for the end-user.

2.2 Languages

The project fundamentally relies on the core languages of the web to ensure cross-browser compatibility and high performance.

2.2.1 JavaScript

JavaScript serves as the core execution engine facilitating the "Logical Layer" of the entire system. By adopting an isomorphic (full-stack) JavaScript approach via the MERN stack, the project achieves seamless data serialization between the client and server [1]. In the context of the TWB project, JavaScript is utilized to manage the complex, recursive state of the website builder, handling intensive asynchronous event loops for drag-and-drop interactions and real-time Document Object Model (DOM) updates without requiring full-page reloads.

2.2.2 HTML (HyperText Markup Language)

Within this system, HTML5 is viewed not merely as static markup, but as the dynamic rendering target for the application's internal state. The builder utilizes HTML5's semantic structural elements to ensure that the dynamically generated code follows modern web standards for SEO and accessibility [2]. Furthermore, the project leverages the HTML5 DOM API to facilitate the "Live Preview" feature, where the virtual layout state is instantly converted into a standardized document structure.

2.2.3 CSS (Cascading Style Sheets)

CSS3 is employed as the "Layout Specification Language," defining the visual boundaries and spatial relationships between components. The project specifically focuses on the "Synthesis of Positioning Paradigms," combining advanced layouts like Flexbox and CSS Grid for structural containment with Absolute Positioning for freeform dragging. This hybrid approach allows the builder to calculate and apply styles dynamically, using CSS as the interpolation layer between the user's spatial input and the final production-ready output [3].

2.3 Frameworks and Libraries

To accelerate development, ensure high performance, and maintain a scalable architectural foundation, several powerful JavaScript frameworks and libraries were incorporated into the system. These specific frameworks were chosen for their proven capabilities in managing highly intensive user interactions and data-handling tasks.

2.3.1 React.js (Declarative UI Library)

React.js acts as the primary "View Orchestrator" for the TWB platform, predicated on a declarative programming paradigm where the user interface is a

direct mathematical function of the application state [4]. Within the Template-Based Website Builder, React is utilized to abstract the complexities of manual DOM manipulation into a structured, state-driven architecture.

The following technical paradigms within React are critical to the system's performance:

- **Component-Based Modularity:** The system decomposes the complex builder interface into isolated, autonomous modules (e.g., Canvas, Sidebar, ComponentProperties). Each component maintains encapsulated logic, facilitating a highly maintainable and scalable codebase while allowing for the granular rendering of specific layout elements [4].
- **Virtual DOM and Diffing Algorithm:** For high-frequency interactive tasks, such as dragging elements across the canvas, the traditional DOM's performance bottlenecks are bypassed using React's Virtual DOM. By employing a sophisticated "Reconciliation" algorithm (Diffing), the system calculates the minimal set of changes required to update the layout coordinates (X, Y), effectively preventing "Layout Thrashing" and ensuring a fluid 60-FPS user experience [4].
- **Unidirectional Data Flow and State Hooks:** To maintain a "Single Source of Truth" for complex website layouts, the project utilizes React Hooks (useState, useCallback, useMemo). This architecture ensures predictable state transitions where changes to a component's property (e.g., opacity or font size) propagate through the system in a unidirectional flow, maintaining data integrity across the editor, properties panel, and live preview [5].

2.3.2 Node.js and Express.js

For the backend architecture, the system utilizes Node.js paired with the Express.js framework to create a highly responsive and scalable server environment.

- **Node.js (Event-Driven Architecture):** Node.js is a cross-platform runtime environment that executes JavaScript code server-side. Unlike traditional multi-threaded server models (like Apache), Node.js operates on a single-threaded, non-blocking, event-driven architecture [5]. This design is exceptionally efficient for handling numerous concurrent Input/Output (I/O) operations. In the website builder, when users save projects, the application transmits large JSON trees (representing the layout schema) to the server. Node.js processes these intense data payloads asynchronously, ensuring the server remains responsive without buffering [5].
- **Express.js (Routing and Middleware):** Paired tightly with Node.js, Express.js acts as a minimal yet powerful web framework. It provides a robust set of features for web applications, primarily acting as the routing layer. In this project, Express is utilized to construct highly secure RESTful APIs [1]. It simplifies the implementation of "Middleware" functions—such as parsing incoming JSON requests, establishing secure cross-origin resource sharing (CORS) policies, and authenticating user requests—facilitating secure and rapid communication between the React frontend editor and the database.

2.4 Development Tools

To facilitate the creation, debugging, and testing of the application, as well as to implement complex interactions safely, several development tools, environments, and specialized libraries were utilized throughout the project lifecycle.

2.4.1 Visual Studio Code

Visual Studio Code (VS Code) is a powerful, lightweight source code editor. It was adopted as the primary Integrated Development Environment (IDE) for its extensive ecosystem of extensions, seamless Git integration, and robust support for the full-stack JavaScript environment [8].

2.4.2 Google Antigravity

Google Antigravity is an advanced agentic AI coding assistant that integrates directly into the development workflow. It was utilized for intelligent code generation, rapid refactoring, debugging complex logic, and assisting in system design, significantly enhancing overall developer productivity [9].

2.4.3 Neovim

Neovim is a highly extensible, terminal-based text editor. It was employed for rapid script editing, server configurations, and efficient shell-based workflows, offering a fast, keyboard-centric environment for quick code modifications [10].

2.4.4 Specifications of the PC used for development:

1. **Processor:** AMD Ryzen™ 5 PRO 5650G @3.9 GHz, Up to @4.4 GHz.
2. **Motherboard:** MSI B450 Gaming Pro Carbon Max WiFi.
3. **Graphics Card:** AMD Radeon™ RX 580 8G Graphics Card.
4. **RAM:** 32GB (3200 GHz).
5. **Disk Storage:** M.2 (256 GB) and SSD (1T).
6. **Operating System:** Arch Linux.

2.4.5 Web Browsers (Google Chrome & Firefox)

Testing the application and its generated layouts across multiple modern rendering engines is critical, as debugging is deeply context-dependent [11].

Google Chrome: Served as the primary browser for development and live debugging, leveraging the powerful Chrome DevTools to inspect DOM elements, monitor network activity, and analyze React component states.

Mozilla Firefox: Utilized to ensure cross-browser compatibility. Its specialized developer tools provide unique advantages, particularly for deeply inspecting and tweaking complex CSS Grid and Flexbox layouts [11].

2.4.6 React Drag and Drop (React DnD)

A crucial aspect of any modern website builder is the ability to select, move, and place elements on a canvas intuitively. React DnD is a powerful set of React utilities that provides high-level abstractions for handling complex drag-and-drop interfaces. Unlike standard HTML5 drag-and-drop APIs, React DnD allows for isolated, state-driven interactions. It makes it possible to seamlessly transfer data (such as dragged component types and unique IDs) across different sections of the workspace. This library forms the backbone of the Hybrid Positioning Engine, ensuring smooth interactions, accurate dropping calculations, and visual updates for the component layout in real time [6].

2.4.7 Version Control (Git & GitHub)

Robust source code management and collaborative capabilities are fundamental to modern software engineering.

- **Git:** Employed as the primary distributed version control system (VCS). It enables tracking granular changes across the codebase, managing

different software versions, and safely reverting to previous states if issues arise.

- **GitHub:** Utilized as the cloud-based hosting platform for the Git repository. It facilitates seamless collaboration, code review via pull requests, issue tracking, and comprehensive project management [12].

2.5 Database Design

Given the nature of website builders, user projects consist of highly nested, varied data structures (like HTML trees and CSS styles) rather than rigid, uniform data blocks. Therefore, a flexible data storage solution is required.

2.5.1 MongoDB

MongoDB is a leading NoSQL, document-oriented database system that stores data in flexible, JSON-like documents (BSON). This structure perfectly aligns with the website builder's core architecture [7]. In this project, every page or template created by a user is essentially represented as a hierarchical JSON tree containing UI elements, node properties, and styles. Using MongoDB allows the backend to store, query, and update these complex page structures natively and efficiently without the need for strict, predefined relational SQL tables. This ensures that massive templates and deeply customized user projects load quickly and can be saved reliably [7].

Core Collections (Database Tables) The system's database schema is designed around three primary collections to manage users, their ongoing projects, and reusable layout templates:

Table 1: User Collection (Manages authentication and user profiles).

Field	Data Type (Length)	Content
name	String (min 2)	The user's full name.
email	String	The user's email address (must be unique).
password	String (min 6)	The user's password, securely hashed via bcrypt.
avatar	String	A URL or path to the user's profile picture.

Implementation (Mongoose Schema):

```
const mongoose = require('mongoose');
```

```
const bcrypt = require('bcryptjs');
const UserSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true,
    minlength: [2, 'Name must be at least 2 characters']
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
```

```
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email address']
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: [6, 'Password must be at least 6 characters']
    },
    avatar: {
      type: String,
      default: ""
    }
  }, {
    timestamps: true
  });

// Hash password before saving
UserSchema.pre('save', async function () {
  if (!this.isModified('password')) return;
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

// Compare password method
UserSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', UserSchema);
```

Table 2: Project Collection

(Acts as the core storage for individual websites created by users).

Field	Data Type (Length)	Content
title	String	The name of the project.
userId	ObjectId	A reference linking the project to its creator (User).
data	Object (JSON)	A flexible JSON object field that stores the entire deeply nested layout structure (UI elements, properties, absolute coordinates) of the website.
isPublished	Boolean	A flag indicating if the website is currently live.

Implementation (Mongoose Schema):

```
const mongoose = require('mongoose');
```

```
const ProjectSchema = new mongoose.Schema({  
  title: {  
    type: String,  
    required: true,  
    trim: true  
  },  
  userId: {  
    type: mongoose.Schema.Types.ObjectId,  
    ref: 'User',
```

```
    },  
    data: {  
      type: Object, // To store page structure (JSON) and drag-and-drop elements  
      default: {}  
    },  
    isPublished: {  
      type: Boolean,  
      default: false  
    }  
  }, {  
    timestamps: true  
  });  
module.exports = mongoose.model('Project', ProjectSchema);
```

Table 3: Template Collection

(Stores pre-designed, ready-to-use component layouts that users can drop into their projects.)

Field	Data Type (Length)	Content
name	String	The unique identifier or name of the template.
components	Array	An array storing the structural JSON of the template elements.

Implementation (Mongoose Schema):

```
const mongoose = require('mongoose');
```

```
const TemplateSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true,
    unique: true
  },
  components: {
    type: Array, // To store the array of component objects
    default: []
  }
}, {
  timestamps: true
});
module.exports = mongoose.model('Template', TemplateSchema);
```